Deep Dive 05 | Chatbot and Gemini Flow

Understand the React request, Flask prompt wrapper, Gemini call and response/error behavior.

React/Vite Edition | Repository-grounded study material | Snapshot: 2026-10-24

1. Request sequence

The chatbot page builds a backend URL from VITE_FLASK_URL or defaults to localhost:5001, then posts JSON containing the user message to /chat. Flask parses JSON, rejects an empty message, creates a Gemini generateContent payload and adds a disaster-education systemInstruction. It posts to Google, extracts the first candidate text and returns JSON.

2. Prompt and data path

- The system instruction frames responses around preparedness, response, recovery and mitigation for learners.
- User content is passed to the provider and may be processed externally; avoid sending unnecessary personal data.
- Gemini candidate content is parsed from nested JSON parts.
- The frontend appends the response to the conversation and displays it.

3. Current configuration

flask/app.py loads GEMINI_API_KEY from the Flask environment and fails startup if missing. The key is incorporated into the provider endpoint. The repository requirements include Flask, Flask-CORS, python-dotenv, gunicorn and requests. The local script runs Flask on port 5001.

4. Failure paths

- No message returns HTTP 400.
- A blocked prompt can return an error based on promptFeedback.blockReason.
- Missing candidate text falls back to a generic provider error.
- The broad exception handler returns exception text to the browser; this can reveal implementation details.
- The current requests.post has no explicit timeout; a slow provider can hold request workers.
- CORS is configured broadly and the direct-run path enables debug mode.

5. Improvements

- Add request timeout and response status checks before parsing JSON.
- Limit message length, request rate and concurrent provider calls.
- Return sanitized error messages with a correlation ID, not raw exception text.
- Restrict CORS to known frontend origins and disable debug in deployment.
- Add abuse controls and conversation privacy notice.
- Keep system prompt server-side and treat model output as untrusted text.

6. Interview explanation

“The React client sends a message to Flask. Flask adds a fixed disaster-education instruction and calls Gemini, then returns the generated text. This separates the model key from the browser. The next reliability steps are timeouts, quota/rate controls, safe error handling, origin restrictions and clear data-privacy expectations.”

7. Study exercise

Follow a message from the send handler to Flask request parsing, payload construction, upstream response parsing and client display. Test empty input, upstream 429, timeout, blocked message, malformed provider JSON and missing candidate content.